

Lecture 11: Abstract Classes and Interfaces

Contracts, default methods, and multiple capabilities

Agenda

1. Abstract classes and abstract members
2. Interfaces in Kotlin
3. Choosing between abstract class and interface

An **abstract class** is an incomplete type: it may declare members without implementations.

- Marked `abstract` — cannot be instantiated directly
- May include concrete state and methods
- Subclasses must implement all abstract members (unless also abstract)

Abstract shape hierarchy

```
1 abstract class Shape {  
2     abstract fun area(): Double  
3     open fun describe(): String =  
4         "area=${area()}"  
5 }  
6  
7 class Circle(val r: Double) : Shape() {  
8     override fun area() = Math.PI * r * r  
9 }  
10  
11 fun main() {  
12     // val s = Shape() // ERROR  
13     println(Circle(2.0).describe())  
14 }
```

Why abstract?

- Capture shared fields/logic for a family of types
- Force subclasses to fill in type-specific behaviour
- Single superclass only (Kotlin/JVM) — plan hierarchies carefully

Common mistake

Using an abstract class when there is no shared state — an interface may be enough.

Quiz 1 — Instantiation

Quiz

Which statement is true?

1. You can write `Shape()` if `Shape` is abstract but has no abstract methods
2. Abstract classes cannot declare concrete methods
3. Abstract classes cannot be constructed with `Shape()`; only concrete subclasses can
4. Abstract members do not need `override` in subclasses

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. You never instantiate an abstract class directly.

Even with zero abstract members, `abstract class` blocks construction. Subclasses use `override` for abstract members.

An **interface** declares a capability / contract.

- No instance fields stored in the classic sense (properties need accessors)
- Methods may be abstract or have default bodies
- A class may **implement multiple** interfaces

Interface basics

```
1 interface Drivable {  
2     fun drive(distance: Int)  
3     fun horn(): String = "beep"    // default  
4 }  
5  
6 class Car : Drivable {  
7     override fun drive(distance: Int) {  
8         println("Driving $distance")  
9     }  
10 }
```

Multiple interfaces

```
1 interface Identified { val id: String }
2 interface Auditable { fun audit(): String }
3
4 class Document(
5     override val id: String,
6     val body: String
7 ) : Identified, Auditable {
8     override fun audit() = "doc=$id len=${body.length}"
9 }
```

Resolving conflicts

```
1 interface A { fun f() = "A" }
2 interface B { fun f() = "B" }
3 class C : A, B {
4     override fun f(): String =
5         super<A>.f() + "+" + super<B>.f()
6 }
```

When two interfaces provide the same default, the class must override and may choose `super<InterfaceName>.member()`.

Abstract class vs interface

Abstract class	Interface
Shared state / constructor logic	Capability / role
Single parent class	Many interfaces
Template method with fields	Default methods without stored fields
Often: interface for contracts; abstract class when you need a partial implementation with fields.	

Quiz 2 — Multiple inheritance of class?

Quiz

Can a Kotlin class extend two abstract classes?

1. Yes, always
2. Yes, if both are in the same file
3. No — only one superclass; many interfaces
4. Only with `data class`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. Only one superclass; multiple interfaces are allowed.

Quiz 3 — Which is valid?

Quiz

Which snippet is valid?

1. `interface I { var x = 1 }`
2. `abstract class A { abstract fun f() }; class B : A() { override fun f() {} }`
3. `class B : A(), C()` where both A and C are classes
4. `val i = Drivable()` for interface Drivable with defaults only

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Concrete class implements abstract members with `override`.

(1) interfaces cannot hold backing-field state like that. (3) two class parents illegal. (4) interfaces are not constructed directly.

Summary

- Abstract classes: partial implementation + forced hooks
- Interfaces: capabilities; multiple; optional default methods
- Override conflicts with `super<T>...`
- Prefer interfaces for API boundaries; abstract classes for shared base state

Looking ahead

Final OOP module lecture: files, mutable collections, and manual serialisation / deserialisation.